

Rapport_Audit_Securite_IoT_Gateway

Auteur : ROMERO Kenaël | JOUHANNET Louis | AVRIL Samuel | RIBEIRO Enzo

Date : 16/07/2026

TP : Sécurité Informatique — Yocto Project

Master 2 ESGI — Joffrey Hérard

Résumé Exécutif

Ce rapport présente les mesures de durcissement appliquées à une image Linux embarquée (passerelle IoT) construite avec le Yocto Project. L'objectif est de verrouiller l'accès à la passerelle en interdisant l'accès root distant, en rendant le système de fichiers immuable, en activant les flags de compilation sécuritaires et en mettant en place un pare-feu strict. Une analyse de vulnérabilités (CVE) est également réalisée.

1. Structure du Layer meta-iot-gateway-hardened

```
meta-iot-gateway-hardened/  
├── conf/  
│   ├── layer.conf  
├── recipes-connectivity/  
│   ├── openssh/  
│   │   ├── openssh_%.bbappend  
│   │   └── files/  
│   │       └── sshd_config  
├── recipes-core/  
│   ├── firewall-config/  
│   │   ├── firewall-config_1.0.bb  
│   │   └── files/  
│   │       └── firewall-config.sh  
│   └── images/  
│       └── iot-gateway-image.bb
```

1.1 conf/layer.conf

Déclare le layer et sa compatibilité avec la branche `scarthgap` :

```
BBPATH .= ":{LAYERDIR}"
BBFILES += "${LAYERDIR}/recipes-*/**/*.bb ${LAYERDIR}/recipes-*/**/*.bbappend"

LAYERDEPENDS_meta-iot-gateway-hardened = "core"
LAYERSERIES_COMPAT_meta-iot-gateway-hardened = "scarthgap"
```

2. Recette d'Image — `iot-gateway-image.bb`

```
inherit core-image
inherit extrausers

IMAGE_FEATURES += "ssh-server-openssh read-only-rootfs"

IMAGE_INSTALL += "iptables firewall-config"

EXTRA_USERS_PARAMS = "\
    useradd -p '$1$sBWvxjvr$FS3NCZJ9lj2Uyim0Rr9yZ/' -G wheel admin; \
    usermod -L root; \
"
```

2.1 Mesures appliquées

Mesure	Mécanisme
Service SSH	<code>IMAGE_FEATURES += "ssh-server-openssh"</code>
Rootfs lecture seule	<code>IMAGE_FEATURES += "read-only-rootfs"</code>
Pare-feu	<code>IMAGE_INSTALL += "iptables firewall-config"</code>
Utilisateur admin	<code>useradd</code> avec hash MD5 du mot de passe <code>admin123</code>
Root verrouillé	<code>usermod -L root</code> (compte root désactivé)

3. Durcissement SSH

3.1 `openssh_%.bbappend`

```
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI:append = " file://sshd_config"
```

3.2 sshd_config

```
PermitRootLogin no
PasswordAuthentication yes
PubkeyAuthentication no
ChallengeResponseAuthentication no
UsePAM yes
X11Forwarding no
PrintMotd no
AcceptEnv LANG LC_*
Subsystem sftp /usr/libexec/sftp-server
```

Explications :

- `PermitRootLogin no` : interdit toute connexion SSH en tant que root.
 - `PasswordAuthentication yes` : autorise l'authentification par mot de passe pour l'utilisateur `admin`.
 - `PubkeyAuthentication no` : désactive l'authentification par clé (cas d'un laboratoire ; en production, on recommande `PubkeyAuthentication yes` et `PasswordAuthentication no`).
 - `X11Forwarding no` : réduit la surface d'attaque.
-

4. Pare-feu

4.1 firewall-config_1.0.bb

```
SUMMARY = "Firewall configuration script for IoT gateway"
LICENSE = "MIT"
LIC_FILES_CHKSUM =
"file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ecda2f7b4f302"

SRC_URI = "file://firewall-config.sh"

inherit update-rc.d

INITSCRIPT_NAME = "firewall-config.sh"
INITSCRIPT_PARAMS = "start 99 S ."

do_install() {
    install -d ${D}${sysconfdir}/init.d
    install -m 0755 ${WORKDIR}/firewall-config.sh
```

```
${D}${sysconfdir}/init.d/firewall-config.sh  
}
```

La recette utilise `inherit update-rc.d` pour installer le script comme service de démarrage (priorité 99, runlevel S).

4.2 firewall-config.sh

```
#!/bin/sh
```

```
iptables -P INPUT DROP  
iptables -P FORWARD DROP  
iptables -P OUTPUT ACCEPT
```

```
iptables -A INPUT -i lo -j ACCEPT  
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT  
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

Règles appliquées :

Règle	Effet
<code>-P INPUT DROP</code>	Bloque tout le trafic entrant par défaut
<code>-P FORWARD DROP</code>	Bloque tout forwarding
<code>-P OUTPUT ACCEPT</code>	Autorise tout le trafic sortant
<code>-A INPUT -i lo -j ACCEPT</code>	Autorise le trafic sur loopback
<code>-A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT</code>	Autorise les connexions déjà établies
<code>-A INPUT -p tcp --dport 22 -j ACCEPT</code>	Autorise SSH entrant (port 22)

5. Durcissement du Compilateur

5.1 security_flags.inc (activé dans `local.conf`)

```
require conf/distro/include/security_flags.inc
```

Flags activés sur l'ensemble du système :

Flag	Rôle
<code>-fstack-protector-strong</code>	Protège contre les buffer overflows en détectant la corruption de la stack canary
<code>-D_FORTIFY_SOURCE=2</code>	Ajoute des vérifications au moment de l'exécution sur les fonctions <code>memcpy</code> , <code>strcpy</code> , etc.
<code>-Wl,-z,relro</code>	Marque les sections ELF en lecture seule après relocation (RELRO)
<code>-Wl,-z,now</code>	Résout toutes les symboles au chargement (BIND_NOW), renforçant RELRO en Full RELRO
<code>-Wl,-z,noexecstack</code>	Rend la stack non exécutable (NX bit)

6. Analyse de Vulnérabilités (CVE)

6.1 Activation dans `local.conf`

```
INHERIT += "cve-check"
CVE_CHECK_REPORT_PATCHED = "0"
```

Le paramètre `CVE_CHECK_REPORT_PATCHED = "0"` permet de ne conserver dans le rapport que les CVE non corrigées.

6.2 Emplacement des rapports

```
tmp/deploy/cve/iot-gateway-image.json
tmp/deploy/cve/iot-gateway-image.txt
```

6.3 Analyse d'une CVE "High" — Exemple avec CVE-2024-XXXX (curl)

Note : Les versions précises des CVE dépendent de la base NVD au moment du build. Les CVE listées ci-dessous sont données à titre d'exemple avec `curl 7.50.0`.

CVE identifiée : CVE-2024-XXXX — Heap buffer overflow dans le parsing HTTP/2 de curl.

- **Score CVSS 3.1 :** 7.5 (High)
- **Composant affecté :** curl < 8.6.0
- **Impact :** Exécution de code arbitraire à distance via un serveur HTTP/2 malveillant.

- **Exploitation** : Un attaquant contrôlant un serveur HTTP/2 peut corrompre la mémoire du client curl.

Stratégie de remédiation :

1. **Mise à jour de la recette** : Modifier `PREFERRED_VERSION_curl` pour pointer vers une version patchée ($\geq 8.6.0$) dans `local.conf` :

```
PREFERRED_VERSION_curl = "8.6.0"
```

2. **Backport du patch** : Dans le cas où la mise à jour majeure n'est pas possible, appliquer le patch de sécurité via `SRC_URI` dans un `.bbappend` :

```
SRC_URI += "file://CVE-2024-XXXX.patch"
```

3. **Vérification** : Relancer `bitbake iot-gateway-image` et consulter le rapport CVE mis à jour pour confirmer que la CVE n'apparaît plus.

7. Procédure de Compilation et de Test

7.1 Compilation

```
cd ~/poky2
bash -c 'source oe-init-build-env build && bitbake iot-gateway-image'
```

7.2 Tests de Validation

Les tests ci-dessous sont effectués dans l'image QEMU (`runqemu qemux86-64 nographic`) après le build.

Test 1 : Connexion root SSH (doit échouer)

```
ssh root@192.168.7.2
# Résultat attendu : Permission denied (publickey,password).
```

Test 2 : Connexion admin SSH (doit réussir)

```
ssh admin@192.168.7.2
# Mot de passe : admin123
# Résultat attendu : connexion établie.
```

Test 3 : Écriture à la racine (doit échouer)

```
touch /test
```

```
# Résultat attendu : touch: cannot touch '/test': Read-only file system
```

Test 4 : Inspection du pare-feu

```
iptables -L -n
```

```
# Résultat attendu :
```

```
# Chain INPUT (policy DROP)
```

```
# ACCEPT      all  --  0.0.0.0/0          0.0.0.0/0
```

```
# ACCEPT      all  --  0.0.0.0/0          0.0.0.0/0          state
```

```
ESTABLISHED,RELATED
```

```
# ACCEPT      tcp  --  0.0.0.0/0          0.0.0.0/0          tcp dpt:22
```

Test 5 : Consultation du rapport CVE

```
cat tmp/deploy/cve/iot-gateway-image.txt
```

8. Configuration Finale (local.conf)

```
# === Durcissement compilateur ===
```

```
require conf/distro/include/security_flags.inc
```

```
# === Analyse CVE ===
```

```
INHERIT += "cve-check"
```

```
CVE_CHECK_REPORT_PATCHED = "0"
```

```
# === Bonus : version vulnérable pour démo ===
```

```
# PREFERRED_VERSION_curl = "7.50.0"
```

```
# PREFERRED_VERSION_openssl = "1.0.2u"
```

9. Configuration des Layers (bblayers.conf)

```
BBLAYERS ?= " \
```

```
  /Users/enzo/poky2/meta \
```

```
  /Users/enzo/poky2/meta-poky \
```

```
  /Users/enzo/poky2/meta-yocto-bsp \
```

```
  /Users/enzo/poky2/meta-iot-gateway-hardened \
```

```
/Users/enzo/poky2/meta-security \  
"
```

10. Conclusion

Toutes les mesures de durcissement demandées ont été implémentées :

- ✓ Layer personnalisé `meta-iot-gateway-hardened`
- ✓ Utilisateur `admin` avec mot de passe, root verrouillé
- ✓ Connexion root SSH interdite
- ✓ Système de fichiers en lecture seule
- ✓ Flags de compilation sécuritaires (`stack-protector`, `FORTIFY`, `RELRO`, `NX`)
- ✓ Pare-feu strict (`INPUT DROP`, seul SSH autorisé)
- ✓ Analyse CVE activée (`cve-check`)
- ✓ Rapport de vulnérabilités généré

Le layer est prêt à être intégré dans tout projet Yocto basé sur `scarthgap` et nécessite uniquement l'ajout des layers `meta-security` et `meta-iot-gateway-hardened` dans `bblayers.conf`.
